Design plan Hartree-Fock Luca Verduin

Program summary

When the program is ran, it asks for an input file. Examples of an input file are given in the **/inputs** folder, this is also where new input files have to be located! More on the input format will be explained in detail later.

The program reads Atom Definitions, Atoms and Settings from the input file.

- -Atom Definitions contain the atomic charge, # electrons and basis set information.
- -Atoms contain references to definitions along with coordinates
- -Settings specify: tolerance, max cycles, output-file, print every

With this the molecule object is created by looping over the atoms and adding the charge and coordinate to arrays used to create the molecule object. (functionality present before the project)

After this the basis is defined by looping over atoms, and adding the defined basis for this atom at the position that the atom is located.

Lastly a subroutine to make the core Hamiltonian is called and following that a subroutine to run the SCF procedure and finish the calculation.

How to run

As mentioned in the program summary, when ran, the program asks for an input file. This is any text file inside **/inputs** (i.e. input H2.txt), which follows the format explained in the Input Format section following this paragraph. If the input does not contain an outfile, the program will print everything to the terminal. If an outfile is specified, then the program will write to this destination inside the **/outputs** folder, as well as print a small summary to the terminal.

Input Format

As mentioned earlier, example inputs are given in **/inputs** folder. Here I will reference to CH4.txt and explain the input.

All lines inside the input file **must** be 'left aligned', as in have no leading spaces.
Comments can be put outside of the blocks by starting the line with '!', this will skip all checks for this line and continue reading the next line.

Input consists of 3 blocks, these are "DEFINE ATOMS" – "/DEFINE ATOMS", "ATOMS" – "/ATOMS" and "SETTINGS" – "/SETTINGS".

Atoms are defined by a symbol (max. 4 characters), charge, n_electrons, a string with angular momenta and a string with exponents. **Important**: for the strings, separate different entries with **ONLY 1 SPACE**! Spaces are used to count how large to allocate arrays to store the definitions.

In the following example for CH4.txt, H is defined with 3 S functions and C is defined with 5 S functions, 3 P functions and 2 D functions.

```
DEFINE ATOMS

H 1.0 1.0 "0 0 0" "0.1 1.0 3.0"
c 6.0 6.0 "0 0 0 0 0 1 1 1 2" "0.10 0.35 1.0 3.0 10.0 0.2 1.0 5.0 1.0"

/DEFINE ATOMS
```

After defining the atoms, the system is inputted quite easily. The entries in the ATOMS block consist of a reference to a definition; This is in the same order (top to bottom) as the definitions. So for CH4 index "1" would reference to H and "2" would reference to c. Each entry is then followed by a string containing coordinates.

```
ATOMS

1 "0.6276 0.6276 0.6276"
1 "0.6276 -0.6276 -0.6276"
1 "-0.6276 0.6276 -0.6276"
1 "-0.6276 -0.6276 0.6276"
2 "0.0 0.0 0.0"

/ATOMS
```

The SETTINGS block then contains some small settings, like tolerance, maxcycles, (optional) outputfile and printevery.
Tolerance is based on the Density matrix (equation 12), maxcycles determines after how many cycles to terminate SCF (if calculation does not converge), outfile specifies the filename where output will be written to. If there is no outputfile, all write statements will write to terminal. Printevery is used to set frequency of print statements in SCF procedure.

For the CH4 example the settings look like this:

```
SETTINGS

tolerance 0.000000001
maxcycles 1000
outfile CH4_OUT.txt
printevery 5

/SETTINGS
```

Module Descriptions

atomtype.f90

Contains the type atom and atom pointer, atom is used to define an atom type, atom_pointer is then used as an actual atom in the system.

(type) atom:

```
Real(8) :: charge
Real(8) :: n_electrons
Integer, allocatable :: angular_momentum(:)
Real(8), allocatable :: exponents(:)
```

(type) atom_pointer:

```
type(atom), pointer :: atom_type
real(8) :: coordinates(3)
```

InOut.f90

Used to read inputs mostly, and write input summary into the output file, the rest of the output is written during SCF.
Contains variables atom_definitions and input_atoms containing atom definitions and actual atoms in the system, as well as variables for the settings (tolerance etc.)

Subroutine getInput():

Asks for a file and reads the input per block. This is achieved by looping over the file, when a block start is read it goes into a new loop where it reads lines until the /block is found.
This creates an array defined_atoms with atom definitions as well as an array with input_atoms with the atoms in the system.

Subroutine generate_molecule():

Generates the molecule from input_atoms, by creating temporary array with charges and temporary array with coordinates. Then calls add_atoms_to_molecule with the charges and coordinates.

Subroutine define_basis()

Loops over atoms in input_atoms and adds for every %angular_momentum(i) and %exponents(i) a function to the basis using add_shell_to_basis() at corresponding coordinates.

HartreeFock.f90

Used to run the actual calculations.
Contains variables like H, F, T, V, S, C, eps and D used in the Hartree-Fock working equations.

Most variables are public in case that additional expansions to the program need these matrices.

Subroutine coreHamiltonian(n_AO, n_occ, molecule, ao_basis):

Allocates S, T, V and computes corresponding integrals.
Allocates F and H (H for core Hamiltonian), computes core Hamiltonian H.
Diagonalizes H and writes orbital energies for the core hamiltonians.
Computes first iteration of D

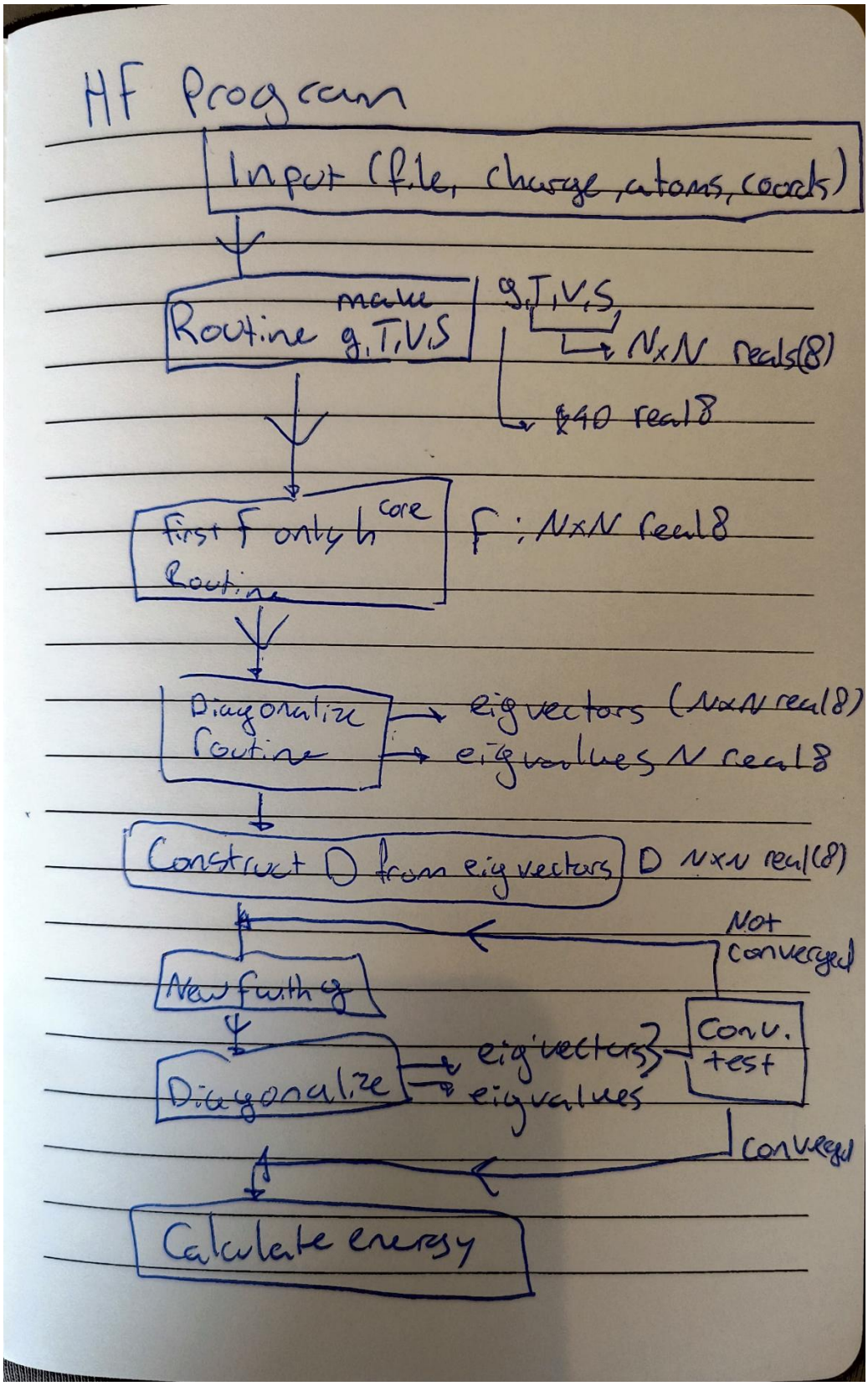
Subroutine SCFprocedure(n_AO, n_occ, max_cycles, tolerance, print_every):

Creates Fock matrix by combining 2-integral terms D and H into F. Diagonalizes F and calculates new D matrix in cycles until the difference in D is converged (formula 12).

Subroutine calculateDensity(C, D, n_occ):

Calculates density matrix.

Subroutine set_output(outfile, outtofile):

Sets logical write_tofile and character output_file in HartreeFock module so other functions can use them without passing them along.

Original Data Flow Diagram